CHAPTER 3

Tuesday: Methods

Yesterday you learned about the Ruby object model and how to make Ruby classes sing and dance for you. Today you're holding all calls to focus on *methods*.

The objects in your code talk to each other all the time. Some languages—such as Java and C—feature a compiler that presides over this chatting. For every method call, the compiler checks to see that the receiving object has a matching method. This is called *static type checking*, and the languages that adopt it are called *static languages*. For example, if you call `talk_simple` on a `Lawyer` object that has no such method, the compiler protests loudly.

Dynamic languages—such as Python and Ruby—don't have a compiler policing method calls. As a consequence, you can start a program that calls `talk_simple` on a `Lawyer`, and everything works just fine—that is, until that specific line of code is executed. Only then does the `Lawyer` complain that it doesn't understand that call.

That's an important advantage of static type checking: the compiler can spot some of your mistakes before the code runs. This protectiveness, however, comes at a price. Static languages often require you to write lots of tedious, repetitive methods—the so-called *boilerplate methods*—just to make the compiler happy. (For example, `get` and `set` methods to access an object's properties, or scores of methods that do nothing but delegate to some other object.)

In Ruby, boilerplate methods aren't a problem, because you can easily avoid them with techniques that would be impractical or just plain impossible in a static language. In this chapter, we'll focus on those techniques.